Initiation au développement

Contrôle moyen

28 Septembre 2022

---

**Durée 2h.** Aucun document n'est autorisé.

Le barème est **indicatif**.

---

## Exercice 1 : Calcul de l'indice de masse corporelle (*6,5 points*)

Le but de cet exercice est de calculer l'Indice de Masse Corporelle (IMC) et d'indiquer à quelle corpulence physique il correspond. L'IMC est égal au poids en kilogrammes divisé par le carré de la taille en mètre :

```
IMC = poids /(taille**2)
```

L'interprétation de l'IMC pour les personnes de 16 ans et plus est la suivante :

- Si l'IMC est strictement inférieur à 18,5, la personne est en *sous-poids*.
- Si l'IMC est supérieur ou égal à 18,5 et strictement inférieur à 25, la personne a un poids *normal*.
- Si l'IMC est supérieur ou égal à 25 et strictement inférieur à 30, la personne est en *surpoids*.
- Si l'IMC est strictement supérieur à 30, la personne est *obèse*.

Ainsi une personne ayant un IMC de 18 est considéré en sous-poids.

On veillera à **ne pas faire de test inutile**.

**1.1 :** Écrire une fonction `corpulence` qui calcule l'IMC en fonction du `poids` et de la `taille` passés en paramètres et affiche l'interprétation de la valeur obtenue. Si l'IMC vaut 18, la fonction affichera

```
sous-poids
```

**1.2 :** Écrire une fonction `saisie_nbpos` permettant la saisie contrôlée d'un flottant et demandant ce nombre jusqu'à ce que ce soit un nombre strictement positif. Cette fonction sera utilisée dans le programme principal pour la saisie du poids et de la taille.

**1.3 :** En utilisant les fonctions `corpulence` et `saisie_nbpos`, écrire un programme principal qui demande à l'utilisateur de saisir un poids (en kilos) et une taille (en m) jusqu'à ce qu'ils soient tous deux strictment positifs et affiche la corpulence correspondante.

## Exercice 2 : Programmation de la fonction Xor (*4 points*)

L'opérateur `Xor` est un opérateur booléen classiquement utilisé en informatique. Sa table de vérité est la suivante :

| var1 | var2 | var1 Xor var2 |
|---|---|---|
| True | True | False |
| True | False | True |
| False | True | True |
| False | False | False |

**2.1 :** Écrire une fonction `Xor()` telle que l'appel de `Xor(var1, var2)` renvoie le résultat de l'opération `var1 Xor var2`.

**2.2 :** Pour engendrer aléatoirement un booléen (et de manière uniforme), on peut tirer aléatoirement (et de manière uniforme) un entier qui vaut 0 ou 1 et ensuite caster ce dernier (avec la fonction `bool()`). Pour un entier, `x`, `bool(x)` vaut `False` si `x` vaut 0 et `True` sinon.

Écrire un programme principal qui engendre aléatoirement 2 booléens et affiche le résultat de l'opérateur `Xor` sur ces valeurs.

## Exercice 3 : Visibilité d'une variable (*2.5 points*)

On considère le code suivant.

```
def tmp(i):
    while (i < 7):
        print("Ohé")
        i+=1
    return 3*i

i = 0
j = 0
cpt=0
while (i < 4):
    j=tmp(i)
    cpt+=1
    i+=1
j = tmp(i)
```

**3.1 :** Combien de fois la fonction `tmp` est-elle appelée ?

- ☐ 2
- ☐ 4
- ☐ 5
- ☐ 7
- ☐ 8

**3.2 :** Combien de fois s'affiche `Ohé` ?

- ☐ 4
- ☐ 7
- ☐ 11
- ☐ 22
- ☐ 25

**3.3 :** Combien vaut `j` à la fin du programme ?

- ☐ 0
- ☐ 4
- ☐ 7
- ☐ 12
- ☐ 18
- ☐ 21
- ☐ 24

## Exercice 4 : Calcul de produits (*7 points*)

**4.1 :** On veut définir une fonction `produit` qui prend en paramètre deux entiers `n1` et `n2` et retourne

— le produit des entiers de $n_1$ à $n_2$, c'est-à dire le produit $n_1 * (n_1 + 1) * (n_1 + 2) * \ldots * n_2$, si $1 \leq n_1 \leq n_2$ ;

— $-1$ sinon.

Par exemple, si la fonction est appelée avec les valeurs 2 et 4, la fonction doit retourner la valeur 24 car 2 * 3 * 4 = 24. Si elle est appelée avec -10 et 10, la fonction doit retourner -1.

Écrire la fonction produit ainsi que sa docstring. Comment accède-t-on à la docstring de cette fonction ?

**4.2 :** Écrire une fonction `fact` qui retourne

— $n!$ quand $n \geq 0$

— $-1$ sinon.

**Rappel** : $n! = 1 * 2 * \ldots * n$ avec $(n >= 1)$ et $0! = 1$

**Important** : Il est possible (et même recommandé) d'utiliser la fonction `produit` de la question précédente. Vous pouvez y faire appel même même si vous n'avez pas répondu à cette question.

**4.3 :** Définir une fonction de tests unitaires pour la fonction `fact` contenant 3 tests unitaires significatifs de la fonction `fact` et appeler cette fonction de test.